Sorting II

Institute of
Algorithms and Theory

Yannic Maus

Abstract Data Types(ADTs)

Abstract Data Type:

- Specification which kind of data should be stored
- Specification which operations should be executeable (e.g. insert(int x))
  - With the semantics of each operation

Data Structure:

- Explicit implementation of an ADT (in some programming language)
- Cost/Runtime of each operation might vary depending on the implementation

Basic Data Structures

- Array
- Linked List, Doubly linked List
- Queue/Stack

Searching:

- Searching in sorted Array: $O(\log n)$ via binary search
- Searching in a (doubly) Linked list is $O(n)$, even if it is sorted.

Warning Data Structures: Caching, Model

ADT: *Which operations do we support*

Implementation of an ADT + Computational Model → Runtime

Warning: *Many data structures based on pointers are fast in the RAM model, but in practice the pointers can be to arbitrary locations in the physical memory and loading the respective elements can cause* ***many cache failures*** *→ slow.*

Priority Queues, Heaps, and Heapsort

Priority Queue

Priorty Queue (ADT):

A set S of elements; each element $x \in S$ has a key $x.key$

- Insert(S,x): Insert x into set S
- Max(S): Return element of S with largest key
- extractMax(S): Returns the element with largest key from S, and removes it
- Increase(S,x,k): Sets the value of the key of x to the new value $k \geq x.key$

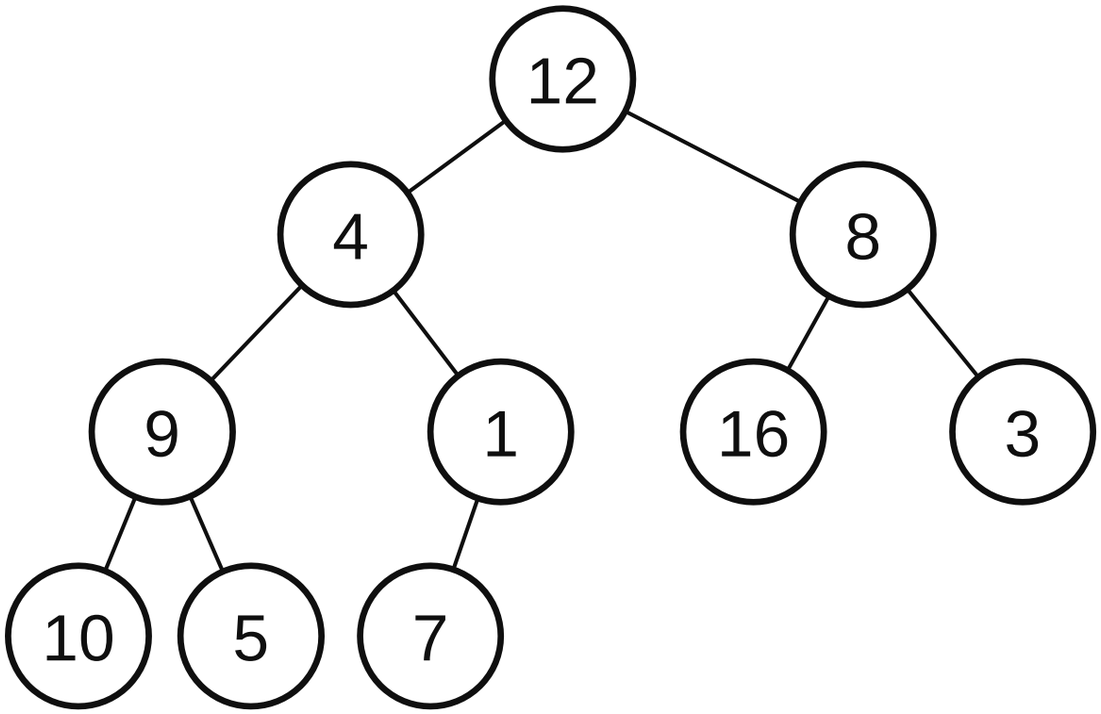
(also exists with Min instead of Max and with slightly different names)

Heap and Max-Heap

Heap is an array visualized as a binary tree:

1	2	3	4	5	6	7	8	9	10
12	4	8	9	1	16	3	10	5	7

(be careful with if array starts at index 0, here first index 1)



- Root = first element
- Children(i): left= $2i$, right $= 2i + 1$
- Parent = $\left\lfloor \frac{i}{2} \right\rfloor$

Max-heap property:
Key of a node is ≥ keys of children

(Min-heap property: Replace ≥ with ≤)

Heap and Max-Heap

Heap is an array visualized as a binary tree:

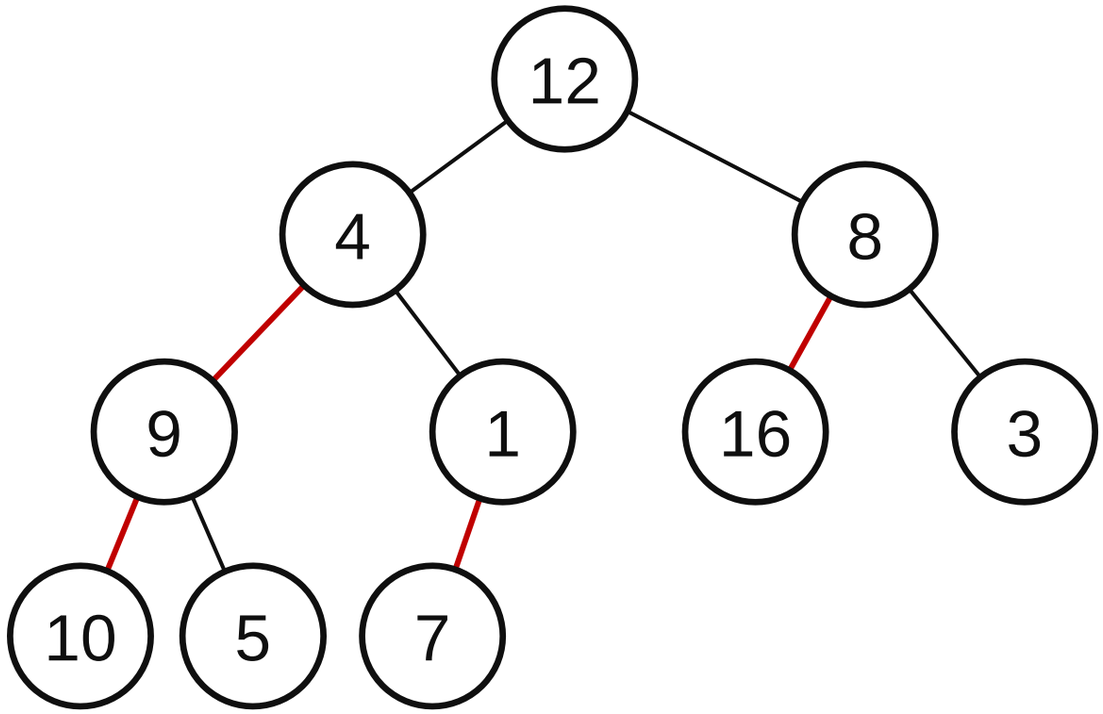
1	2	3	4	5	6	7	8	9	10
12	4	8	9	1	16	3	10	5	7

(be careful with if array starts at index 0, here first index 1)

- Root = first element
- Children(i): left= $2i$, right $= 2i + 1$
- Parent = $\left\lfloor \frac{i}{2} \right\rfloor$

Max-heap property:
Key of a node is ≥ keys of children

(Min-heap property: Replace ≥ with ≤)



(Max-heap property violated)

(Max-)Heap operations

Max-Heap implements a priority queue

Max-heap=Heap + max-heap property

Max-heap property:
Key of a node is ≥ keys of children

Max-Heap operations:

- Operations of a priority queue (insert, Max, extractMax, increase), and
- Build_max_heap: procudes a max-heap from an (unsorted) array
- Heapify(A,i): corrects a **single violation** of the max-heap property

(we will often drop the max-, when speaking of a max-heap)

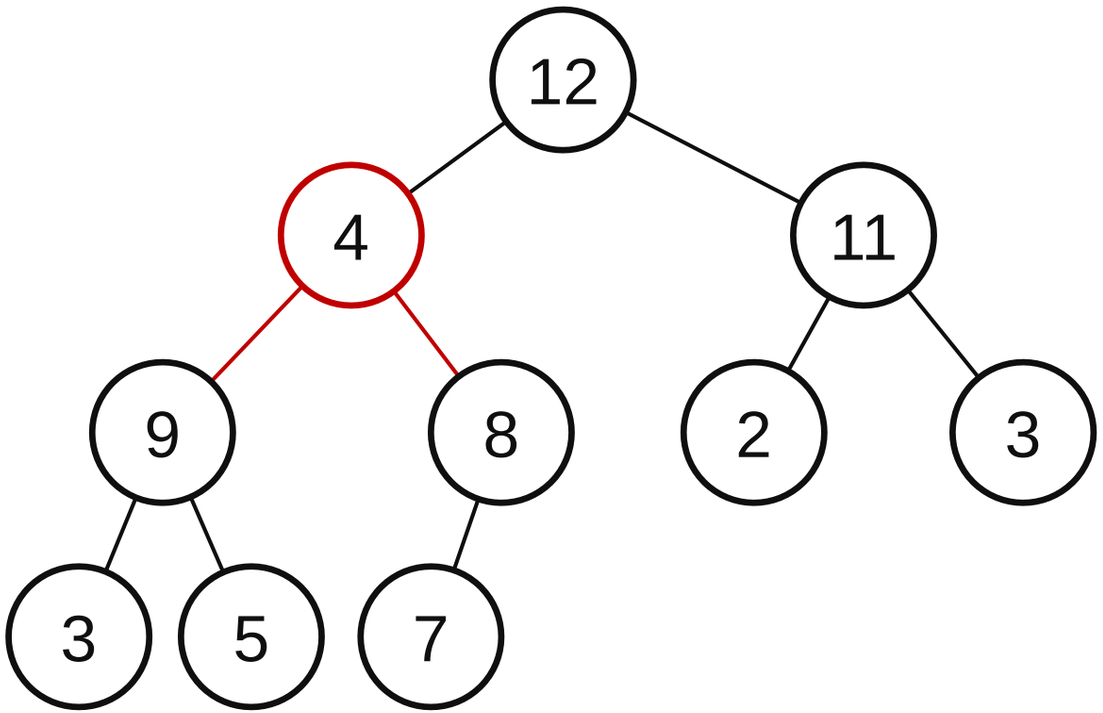
Heapify(A,i): Idea

Input: array A, and index i at which the max-heap property is violated
Pre-condition: Trees rooted at $left(i)$ and $right(i)$ are max-heaps

Output: Heap with restored max-heap property

Idea: Let the element with the error "sink" through the heap to the bottom

index $i = 2$



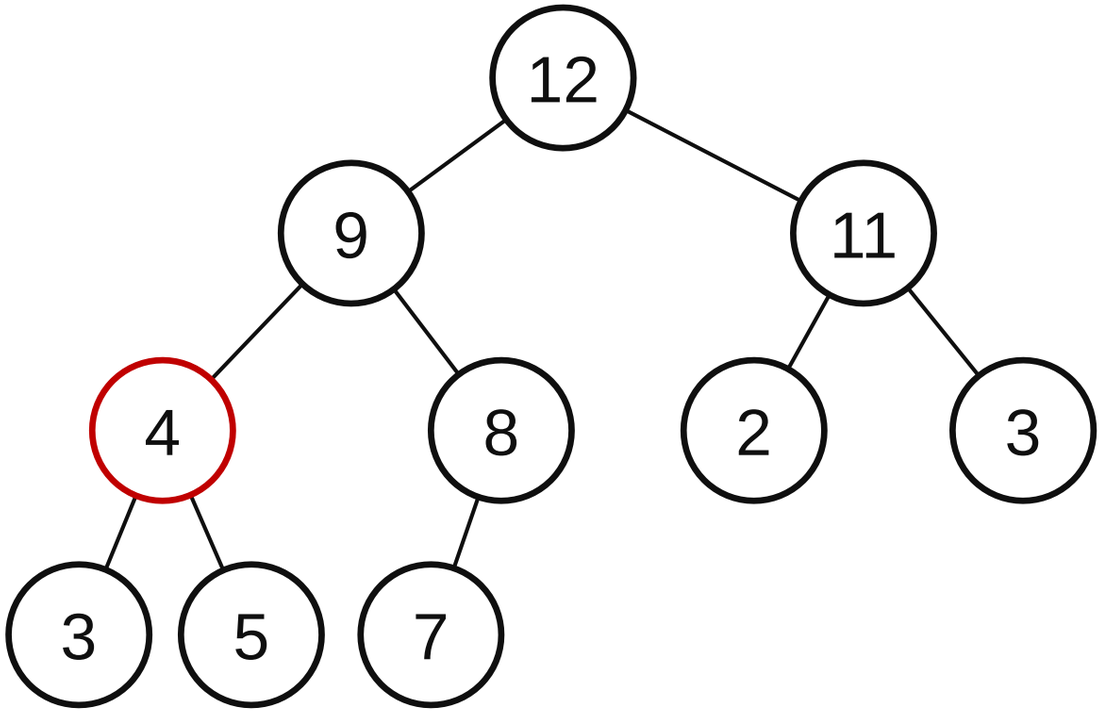
Heapify(A,i): Idea

Input: array A, and index i at which the max-heap property is violated
Pre-condition: Trees rooted at $left(i)$ and $right(i)$ are max-heaps

Output: Heap with restored max-heap property

Idea: Let the element with the error "sink" through the heap to the bottom

index $i = 4$



Heapify(A,i): Idea

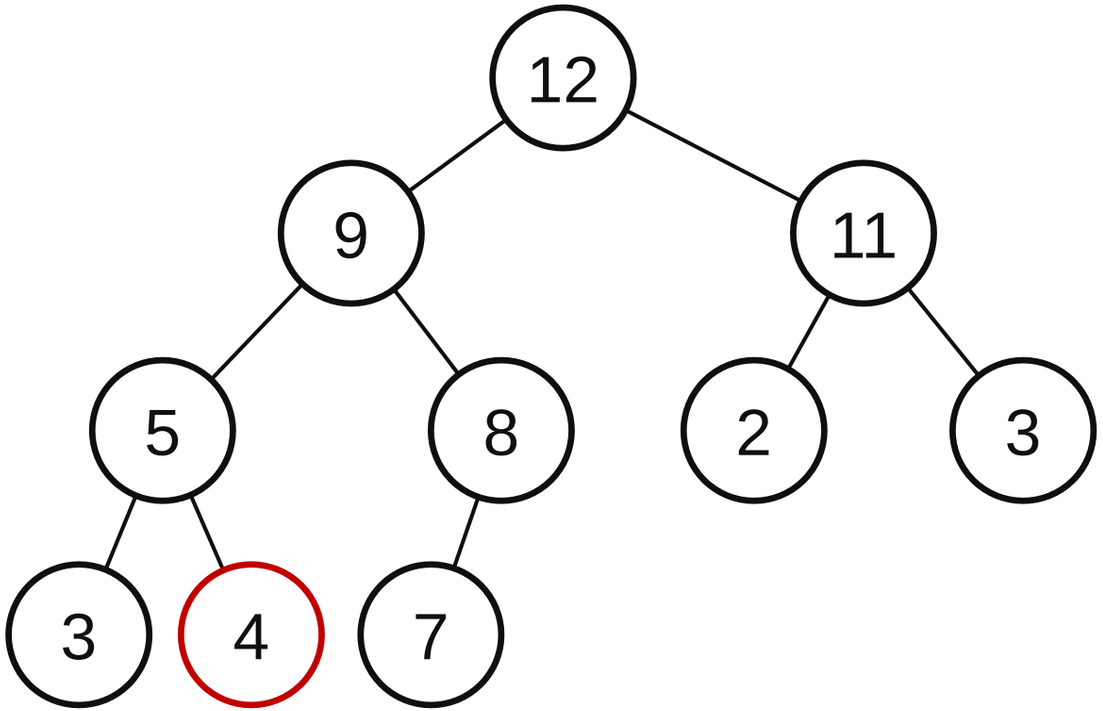
Input: array A, and index i at which the max-heap property is violated
Pre-condition: Trees rooted at $left(i)$ and $right(i)$ are max-heaps

Output: Heap with restored max-heap property

Idea: Let the element with the error "sink" through the heap to the bottom

index $i = 9$

→ no more violation



Heapify(A,i): Pseudocode

```
HEAPIFY(A, i):
    largest = i
    if 2i < length(A) and A[2i] > A[i]:   largest = 2i
    if 2i+1 < length(A) and A[2i+1] > A[largest]: largest = 2i+1
    if largest ≠ i:
        swap(A[i], A[largest])
        HEAPIFY(A, largest)
```

Heapify: Correctness

Base Case (Height = 0): If the subtree rooted at node i has **height 0**, then the node itself is trivially a heap. **Heapify(A, i)** does nothing, and the heap property holds.

Inductive Step ($h \rightarrow h+1$):

Assumption: Suppose that for any node at height h, **Heapify(A, i)** correctly maintains the heap property.

- Node i violates the heap property because its key is **smaller** than at least one of its children.
- The **largest child** is swapped with node i to restore the heap property at this level.
- By the **induction hypothesis**, recursively applying **Heapify** to the swapped child (which is at a lower height) will correctly restore the heap property.

Heapify: Runtime

Intuition: The heap is a binary tree → depth at most $\log n$, sinking an element at most $\log n$ times.

Runtime:

- Ignoring the recursive call, one execution of HEAPIFY takes $c = O(1)$ time
- In each recursion call the index i is (at least) doubled; the index cannot exceed n.

$$i \cdot 2^{\#RecursionCalls} \leq n$$

- **Runtime:** $O(\log n)$ (less if you call the procedure for a vertex close to a leaf)

Build_max_heap

Build_max_heap: procudes a max-heap from an (unsorted) array

Input: Arbitrary array A

Output: Max-heap with values from A

Idea: Build heap from bottom (in the tree) to top

```
Build_max_heap(A):
    For i=n/2 down to 1
        HEAPIFY(A, i)
```

Why do we start at $i = n/2$ and not at $i = n$?
Trees rooted at leaves (single elements) are already max-heaps!

Build_max_heap: Correctness

```
Build_max_heap(A):
    For i=n/2 down to 1
        HEAPIFY(A, i)
```

Proof by induction:

- **Base case:** subtrees rooted at leaves satisfy the heap property
- **Induction step (slowly increasing the height):**
  Each time you call heapify(A,i) you meet the precondition:
  left-child(i) and right-child(i) are proper max-heaps of smaller height. After heapify the max-heap property holds for the subtree rooted at A[i].

At the end the max-heap property holds for the subtree rooted at A[1], i.e., for the whole heap.

Build_max_heap: Correctness & Runtime

```
Build_max_heap(A):
    For i=n/2 down to 1
        HEAPIFY(A, i)
```

Runtime: $\frac{n}{2} \cdot O(\log n) = O(n \cdot \log n)$

More careful analysis of the runtime:

- On level 1 of the tree, we only have one call of Heapify(A,1) for the root of the tree, depth $\log n$:
- On level 2 of the tree, we have two calls of Heapify, taking recursion depth $\log n$-1 each
- On level 3 of the tree we have four calls of Heapify taking recursion depth $\log n$-2 each
- Looking at this bottom up: We have $\approx n/2^j$ nodes with recursion depth j each

$$\sum_{j=1}^{\log n} c \cdot \frac{n}{2^j} \cdot j = c \cdot n \cdot \sum_{j=1}^{\log n} \frac{j}{2^j} \leq c \cdot n \cdot \sum_{j=0}^{\infty} \frac{j}{2^j} = 2 \cdot c \cdot n$$

Heapsort: Algorithm

Idea for sorting an Array A:

1. Build Heap A
2. Find largest element, swap to end of array
3. Decrease heap size by one
4. Heapify (violation can only be at the first position)
5. Repeat (go to Step 2)

```
HEAPSORT(A):
    build_max_HEAP(A)
    for i = n down to 1:
        swap(A[1], A[i])
        Discard A[i] from the heap
        HEAPIFY(A, 1)
```

Heapsort: Runtime

Total runtime: $O(n \cdot \log n)$

O(n)

n iterations

O(log n)

```
HEAPSORT(A):
    build_max_HEAP(A)
    for i = n down to 1:
        swap(A[1], A[i])
        Discard A[i] from the heap
        HEAPIFY(A, 1)
```

Lower bound for Sorting

Can we sort better than the $O(\mathrm{n} \cdot \log n)$ runtime?

Comparison Model

- All items we want to sort (e.g. numbers) are black box abstract data types
  - The only internal access that we have to them is that we can **compare two items, $\boldsymbol{x}, \boldsymbol{y}$:** result "$x \leq y$" holds, or ($x \leq y$ does not hold)

- **Time:** #comparisons
  - (besides the comparisons you can do all kind of stuff and don't have to pay for it)

Examples:

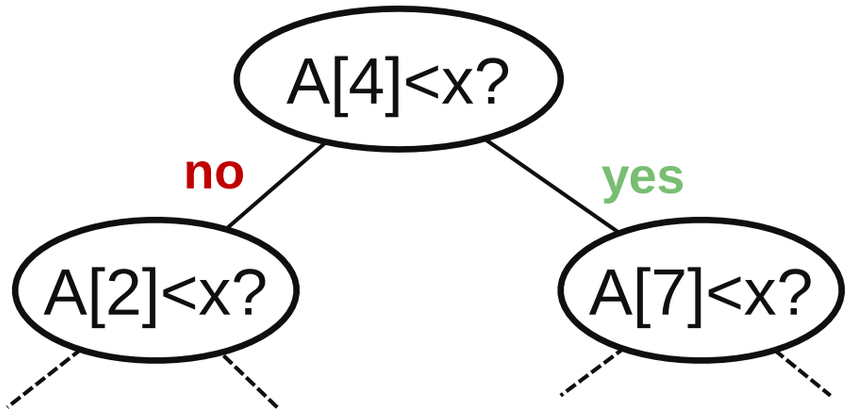
- MergeSort, Heapsort: $O(n \cdot \log n)$ comparisons (worst case)
- Quicksort, Insertion Sort, Selection sort: $O(n^2)$ comparisons (worst case)

Decision Tree

Any comparison-based algorithm can be viewed as a **decision tree** with all comparisons and their outcome, and the resulting answer at the end

Internal node: binary decision (comparison)

Leaf: returning an answer

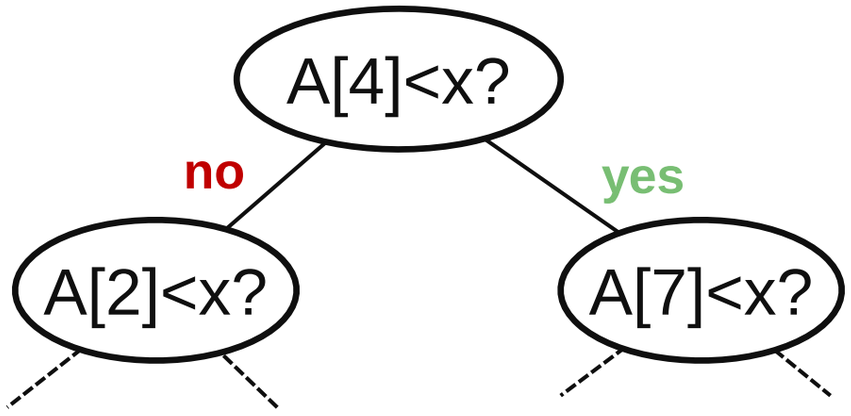


(small part of the huge tree)

- **Execution of an algorithm:** root to leaf path
- **Runtime of the execution:** length of that path
- **Worst case runtime:** Height of the tree

Decision trees for lower bounds

How fast can you solve your problem = How shallow can you make your tree?



(small part of the huge tree)

The decision tree can be very large, often exponential size. It contains all possible executions of an algorithm (usually for a fixed input length). That's not how you want to represent an algorithm (algorithm should have finite code independent from the size of the input n!), but good for analysis!

Searching in structured data

Searching:

- n preprocessed items (can do whatever you want to do, store them in some fancy data structure …)
- **Goal:** find a given element x

Theorem: Finding a given element takes $\Omega(\log n)$ comparisons.

- There are trees (aka search algorithms) that take $O(n)$ steps (linear search)
- What's the best among ALL algorithms?

Proof:

Decision tree is **binary**, must have **at least n leaves**, one for each answer (many leaves may have the same answer, depending on the algorithm)

➔ Height has to be $\Omega(\log n)$

Sorting an element

Theorem: Sorting n elements needs $\Omega(n \cdot \log n)$ comparisons.

Proof:

Consider the **binary** decision tree describing some algorithm.

It has a leaf for each possible output, e.g., one output could be (if n elements stored I array)

A[4]	**A[3]**	**A[0]**	**A[7]**	**A[2]**	**…**	**A[n]**	**A[1]**

#leaves≥#possible answers= $n!$

$$\text{Height} \geq \log_2 n! = \log\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right) = \Omega(n \log n)$$

Stirlings formular:

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

Linear-Time Sorting

Not in the comparison model! Back to the RAM Model

Model assumptions: RAM + Integers

Assumptions:

- n keys for sorting are integers
- Non-negative numbers in some range $\{0, \ldots, k-1\}$
- Each number fits into one word (see RAM definition)

Possible things one could do:

- Comparison $O(1)$ time
- Adding/subtracting keys
- much more …

Linear-Time sorting

Theorem: There is a deterministic algorithm to sort n Integers from a range of size $k = O(n)$ in $O(n)$ time under the previous assumptions.

State of the art: There is a randomized algorithm to sort n integers in $O\left(n \cdot \sqrt{\log \log n}\right)$ time with high probability. (beyond this course)

Conjecture (linear time sorting):
There is a randomized algorithm to sort n integers in $O(n)$ time with high probability.

With high probability: with probability at least $1 - 1/n$ on instance of size n

Counting Sort Version 1

Theorem (counting sort): There is a deterministic algorithm to sort n integers from a range of size k in $O(n+k)$ time under the previous assumptions.

Counting Sort:

- Initialize array C of length k with all zeros
- Go through the input array and count the occurrences of each number in C:

  C[A[i]]++

- Write down the output depending on the counts by iterating

This is good enough if we only need to sort integers. But usually, these integers are just some keys of some abstract data typ

Solution: *Use lists in the counting array (next slide)*

Counting Sort Version 2

Idea: Use an array C of size k,
in which the i-th entry is a list (to be filled) with all elements of value i

```
COUNTING_SORT(A, k)
1. Create an array C of size k, where each entry is an empty list
2. for i=0 to n-1
       Append object A[i] to C[A[i].key]
3. index = 0
4. for i from 0 to k-1:
       for each element x in C[i]:
           B[index] = x
           index++
5. Return B
```

Linear-Time sorting: Radix Sort

Theorem: There is a deterministic algorithm to sort n Integers from range $\boldsymbol{k = n^{O(1)}}$ in O(n) time under the previous assumptions.

- Integers represented by base b => $d = \log_b k$ digits per number

Algorithm:

```
For i=1 to d //sorting bit by bit, start with least significant bit
    Sort by bit i in bit presentation using counting sort
```

You can extract the respective digit in constant time aka get the key for counting sort.

Runtime: $O(d(n+b) = O(\log_b n \cdot (n+b)) = O\big(\log_n k \cdot (n+n)\big) = O(n)$

$b = n$, $k \leq n^c$, for constant c

Radix Sort: Correctness I

Definition: A sorting algorithm is **stable** if, for any two elements $A[i]$ and $A[j]$ such that $A[i] = A[j]$ and $i < j$ in the original array, their relative order remains the same in the sorted array.

The key property that ensures Radix Sort's correctness is that the sorting algorithm used for each digit (**Counting Sort**) is **stable**.

Radix Sort: Correctness II

IH: After processing the i-th bit, elements are sorted according to the bit representation of the first i bits

Base Case (Sorting by the Least Significant Digit)

When sorting by the **least significant digit** (LSD), the numbers are grouped correctly based on this digit.

Inductive Step (Sorting by Next Higher Digit)

Assume that after sorting the first i digits (from right to left), the numbers are **partially sorted** in the correct order up to the i-th digit.

When sorting by the $(\boldsymbol{i}+\mathbf{1})$**th** digit, numbers are grouped based on that digit, but the stable sort ensures that numbers with the same $(i+1)$th digit retain the relative order they had from sorting the previous k digits.

By induction, after sorting all d digits, the entire list is sorted correctly.

Sorting Algorithms: Further properties

Definition: A sorting algorithm falls into the **adaptive** sort family if it takes advantage of existing order in its input. It benefits from the **presortedness** in the input sequence – or a limited amount of disorder for various definitions of measures of disorder – and sorts faster.

(vague definition)

Definition: An sorting algorithm is in-place, if it **use only a constant amount of extra memory (O(1))** beyond the input array. It rearranges the elements **within the input data structure itself** without needing extra arrays or significant memory overhead.

Properties of Sorting Algorithms

Sorting Alg.	In-place	stable	adaptive	Worst case
Insertion sort	Yes	Yes	Yes	$O(n^2)$
Selection sort	Yes	No*	No	$O(n^2)$
quicksort	Yes	No	No	$O(n^2)$
mergesort	No	Yes	No	$O(n \log n)$
heapsort	Yes	No	No	$O(n \log n)$
Counting sort	No	Yes	No	$O(n + k)$
Radix sort**	No	Yes	No	$O(n)$

*: Not stable for our implementation. But there are stable implementations of SelectionSort
**: Bound on the range of values required for the runtime